# SuperEncoder 调优总结

## Results

### 基线

| | IC | ICIR | Short | Long | LS | ShortIr | LongIr | LSIR | ShortHS | LongHS |
|---|---|---|---|---|---|---|---|---|---|---|
| 2019 | 0.17269 | 1.96897 | -0.46984 | 0.38649 | 1.55169 | -7.06789 | 10.13474 | 16.45177 | 0.06201 | 0.06474 |
| 2020 | 0.15030 | 2.49546 | -0.48235 | 0.41857 | 1.66814 | -6.28426 | 9.02282 | 16.66394 | 0.06200 | 0.06507 |
| 2021 | 0.12880 | 2.41484 | -0.46436 | 0.42179 | 1.58992 | -8.48312 | 8.18676 | 20.36490 | 0.05887 | 0.05758 |
| all | 0.15063 | 2.12289 | -0.47223 | 0.40883 | 1.60275 | -7.09386 | 8.95168 | 17.56004 | 0.06096 | 0.06246 |

### SuperEncoder

| Year | IC | ICIR | Short | Long | LS | IR_Short | IR_Long | IR_LS | HS_Short | HS_Long |
|---|---|---|---|---|---|---|---|---|---|---|
| 2019 | 0.1743 | 2.3221 | −0.4500 | 0.3750 | 1.4453 | −7.0852 | 11.9531 | 18.0481 | 0.0658 | 0.0672 |
| 2020 | 0.1457 | 2.3959 | −0.4254 | 0.3841 | 1.3619 | −5.4972 | 9.5221 | 14.1154 | 0.0665 | 0.0675 |
| 2021 | 0.1328 | 2.2598 | −0.4454 | 0.4496 | 1.5549 | −8.7063 | 7.2427 | 18.1486 | 0.0639 | 0.0631 |
| **all** | **0.1509** | **2.2374** | **−0.4404** | **0.4025** | **1.4527** | **−6.7829** | **8.6385** | **16.5457** | **0.0654** | **0.0659** |

## 一、模型架构演进

### 起点：FactorAutoencoder

- 输入：每只股票的原始因子向量（~1128 维）
- 编码器 MLP → latent → 预测头输出 alpha
- 每只股票独立处理，无跨截面信息

### 升级：SuperEncoder（Autoencoder + Cross-Sectional Transformer）

**思路**：先对每只股票独立压缩因子，再让同一日期所有股票在 latent 空间互相 attention，最后做预测。

```
Per-stock:  Input (1128) → Encoder MLP → z (64-dim latent)
Per-date:   [z_1, z_2, ..., z_N] → LayerNorm → TransformerEncoder → h
            enhanced = sigmoid(cs_gate) × h + z
Per-stock:  enhanced → Predictor → prediction
            z        → Decoder → reconstruction (正则)
```

**符号说明：**

- `z`（latent）：每只股票经 Encoder 压缩后的 64 维向量，只包含该股票自身的因子信息，不含任何截面交互
- `h`：所有股票的 `z` 经 Cross-Sectional Transformer 后的输出，每只股票的表示已融合了同日其他股票的信息（行业结构、相对定价等）
- `enhanced`：门控融合后的最终表示，`sigmoid(cs_gate) × h + z`，模型自行学习 CS 信号的贡献比例

- Encoder 在 latent 空间做 attention，计算量远低于在原始 1128 维做 attention
- Autoencoder bottleneck 提供强正则，防止 1128 个共线因子过拟合
- Cross-sectional attention 学习相对定价、行业结构、拥挤效应

---

## FineTuning

### 核心矛盾：

反复实验发现，SuperEncoder 最大的风险是**过拟合截面结构**. 模型在训练期学到了特定年份的行业/风格关系，但在测试时这些关系可能会漂移，导致 IC 波动增大、ICIR 和收益不稳定。

### 关键调整路径：

**1. Latent dim：256 → 64**

- 初始实验 latent_dim=256 时，encoder 容量过大，CS attention 层面过拟合严重
- 逐步从 256 → 128 → 64，发现 **64 维已足够编码 ~1128 个因子的有效信息**（因子间大量共线，有效维度远低于表面维度）
- 对应 CS heads 从 8 → 4（保持每个 head 16 维：64 / 4 = 16），每 head 表达力不变

**2. CS Attention 影响比预期大 → 加 Gate**

- 对比实验发现：CS attention 层对最终结果的影响远超 encoder/predictor 的超参选择
- 在某些 snapshot 上，CS attention **反而降低** IC
- 解决方案：加入可学习 `cs_gate`，让模型自行决定"用多少 CS 信号"
- 训练结束后检查可诊断该 snapshot 的跨截面信号是否有效

```
enhanced = sigmoid(cs_gate) * h + z
```

- `cs_gate` 是一个可训练的标量参数，初始化为 `0.1`
- `sigmoid(0.1) ≈ 0.52`，初始时 CS 信号贡献约 50%，但训练早期通过 loss 反传会迅速调整
- 实际上初始化为接近 0 的值（负数）可以让模型先完全依赖 autoencoder，再逐步引入 CS 信号
- 训练结束后观察 `sigmoid(cs_gate)` 的值可以诊断：**CS 信号对该快照是否真的有帮助**
  - 接近 1：模型认为跨截面信息有价值
  - 接近 0：模型放弃了 CS attention，退化为纯 autoencoder

**与直接去掉 CS Transformer 相比，** gate 的优势是：模型自己学习用多少。这对于不同年份、不同市场状态的泛化能力更强。

**3. Dropout 0.1 → 0.5**

- 早期 dropout=0.1 时，训练 loss 下降快但验证集表现不好，典型过拟合
- 逐步提高 dropout：0.1 → 0.2 → 0.3 → 0.5
- 和 BatchNorm 配合不冲突：BN 提供归一化，Dropout 提供随机稀疏化，两者互补

**4. 针对 ICIR 优化的具体调整**

为了直接优化 IR，做了以下改动：

**损失函数：IC Consistency Loss**

**问题**：MSE 损失优化均方误差，与 ICIR 没有直接关系。模型可以在 MSE 意义上表现好，但 IC 波动很大。

**方案**：

```
ic_loss = -mean(IC_per_date) + std(IC_per_date)
```

第一项最大化平均 IC，第二项最小化 IC 方差，合起来直接优化 ICIR。

#### 总损失

$$L_{\text{total}} = \underbrace{L_{\text{MSE}}}_{w=1.0} + \underbrace{0.1 \times L_{\text{recon}}}_{\text{重建正则}} + \underbrace{0.5 \times L_{\text{IC}}}_{\text{IC一致性正则}}$$

- $L_{\text{MSE}}$：预测头输出 vs 真实收益的均方误差，权重 **1.0**（隐式，始终是主项）
- $L_{\text{recon}}$: 防止 latent 退化
- $L_{\text{IC}} = -\mathbb{E}_t[IC_t] + \sigma_t(IC_t)$：IC 一致性损失，权重 **0.5**（`ic_loss_weight`）

MSE 是绝对主导，IC loss 是辅助正则（MSE 的一半），重建损失更小（1/10）。

#### Early Stopping：改用 ICIR